RISC-LLM-TI

An Ontology-Governed Knowledge-Graph Pipeline for Digital-Asset Threat Intelligence

Technical Report for the PhD Management Committee

How the pipeline works, what it contributes, and why it matters to institutions running digital assets on public blockchains.

> *RISC-LLM-TI uses a deliberately constrained large language model to convert unstructured threat intelligence into an ontology-governed knowledge graph that accumulates institutional memory, derives cross-source relationships invisible to any single source, and quantifies digital-asset exposure with full evidential traceability — combining the language model's understanding with the graph's explainable reasoning, and overcoming the limitations of each used alone.*

1. Executive Summary

Institutions that issue and custody tokenised assets on public blockchains face a threat landscape that is shared, composable, and fast-moving. Intelligence about it arrives as unstructured prose — incident post-mortems, auditor disclosures, CVE advisories, on-chain forensics. The operational question is never "what does this one report say," but "given everything known across all sources, what is my exposure, why, and what must I fix."

A language model alone cannot answer this durably: it has no persistent memory across documents, no auditable evidence trail, and no guarantee it has not silently omitted a critical fact. This work addresses that gap with a pipeline that uses the language model **only** for what it is good at — reading messy prose into candidate facts — and a **knowledge graph** for everything that requires memory, structured reasoning, exposure computation, and explanation.

Four contributions: (1) schema-constrained, standards-grounded graph construction; (2) entity resolution treated as a first-class problem; (3) graph-quality metrics used as construction-failure detectors; (4) downstream-task fidelity (e.g. "who ran the attack") as the real measure of success. Each is motivated by concrete, reproducible failure modes observed in the system itself.

2. The Problem, and Why a Pure-LLM Approach Fails

On a public chain, a tokenised deposit inherits the risk of the bridge it depends on, the consensus of the chain beneath it, and the security of every protocol it touches. The attack surface is **transitive** — a property a list of CVEs or a spreadsheet of incidents fundamentally cannot represent, but a graph naturally can.

If one simply pasted documents into a language model and asked, the result would be fluent but unusable for a regulated institution: answers vary between runs, nothing is traceable to evidence, there is no memory between sessions, and there is no defence against confident omission. A capital or custody decision cannot rest on an answer that cannot be reproduced or defended to an auditor.

3. How the Pipeline Works

The pipeline runs seven stages. The language model is invoked at exactly one of them (extraction); every other stage is deterministic.

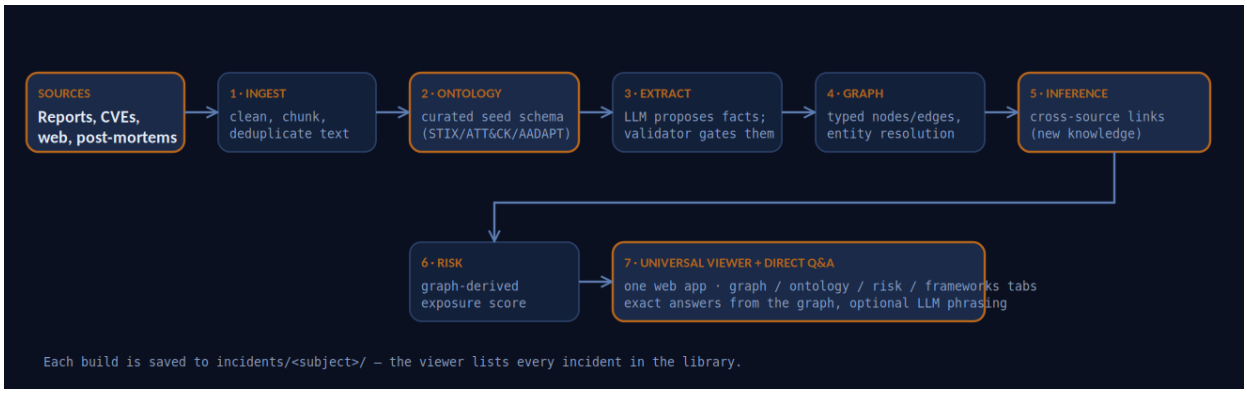

Figure 1 — The seven-stage pipeline. Accented boxes are deterministic; only Stage 3 invokes the language model.

Stage 1 — Ingestion

Sources (local incident reports, or web-scanned primary post-mortems and auditor disclosures, prioritised over secondary news) are cleaned, de-duplicated, and chunked into passages.

Stage 2 — Ontology (the type system)

A curated, standards-grounded ontology fixes **what can exist**. The entity model follows STIX object types; techniques map to MITRE ATT&CK and AADAPT; vulnerabilities use CVE/CWE. Under strict mode the language model may not extend this schema — it is the authoritative contract every fact must satisfy.

Stage 3 — Extraction (the only LLM stage)

For each passage the model proposes typed entities and relationships, each carrying the evidence sentence and a confidence value. A validator then **rejects anything off-schema** before it can enter the graph. This is the "LLM proposes; ontology validator decides" principle that converts free-form generation into auditable, typed facts.

Stage 4 — Graph construction & entity resolution

Accepted facts become typed nodes and edges. Entity resolution merges aliases of the same real-world entity (a recurring failure point treated as research in Section 6).

Stage 5 — Inference (new knowledge)

Deterministic graph algorithms derive relationships no single source stated — shared infrastructure, transitive attack paths, control gaps. Each derived edge records the derivation that produced it.

Stage 6 — Risk

Exposure scores are computed from graph structure (e.g. damped path counts from threat actors to an asset), every contributing factor traced to specific edges, so the number is reproducible and auditor-defensible.

Stage 7 — Universal viewer & direct Q&A

One web application loads any stored incident from the library and presents Graph, Ontology, Knowledge, Risk, and Framework-mapping views. Natural-language questions are answered by **direct**

graph query — exact entity names, instantly, with optional language-model phrasing that may not rename or invent anything.

4. Ontology and Knowledge Graph: The Core Distinction

The distinction between the two central artefacts is frequently muddled; stating it precisely is essential.

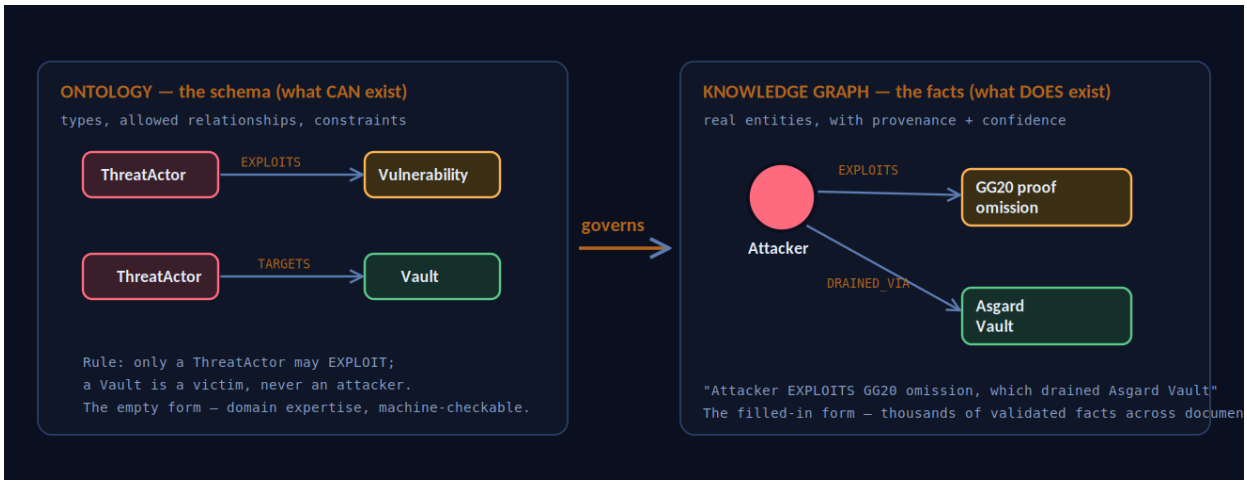

Figure 2 — The ontology is the empty form (schema); the knowledge graph is the filled-in form (facts). The ontology governs what the graph may assert.

The ontology is the schema of what can exist — the types, the permitted relationships, and the constraints (only an actor may exploit; a vault is a victim, never an attacker). It encodes human domain expertise in a machine-checkable form and contains no incident-specific facts.

The knowledge graph is that schema populated with real facts extracted from documents, each node and edge carrying provenance — the source and sentence that support it. **The relationship between them is governance:** the ontology is the active validation boundary through which every candidate fact must pass, which is exactly what makes the graph auditable rather than a free-form dump.

5. Why the Knowledge Graph Is Necessary — The Value Proposition

The graph is not a presentation layer. It earns its place through three properties a language model cannot provide, each mapping to a concrete institutional need.

5.1 Persistent, accumulating institutional memory

A human analyst's understanding leaves when they leave. The graph persists it as structured, queryable knowledge and **accumulates** across every incident ingested. The model is invoked once per document; the resulting knowledge is reused indefinitely. This is the “expensive model once, reuse the knowledge forever” economics — something a language model, which forgets every conversation, structurally cannot offer.

5.2 Cross-source inference — making the invisible visible

This is the core of the contribution. Any single report conveys only what its author knew. The graph derives knowledge no single source states, by composing facts across documents.

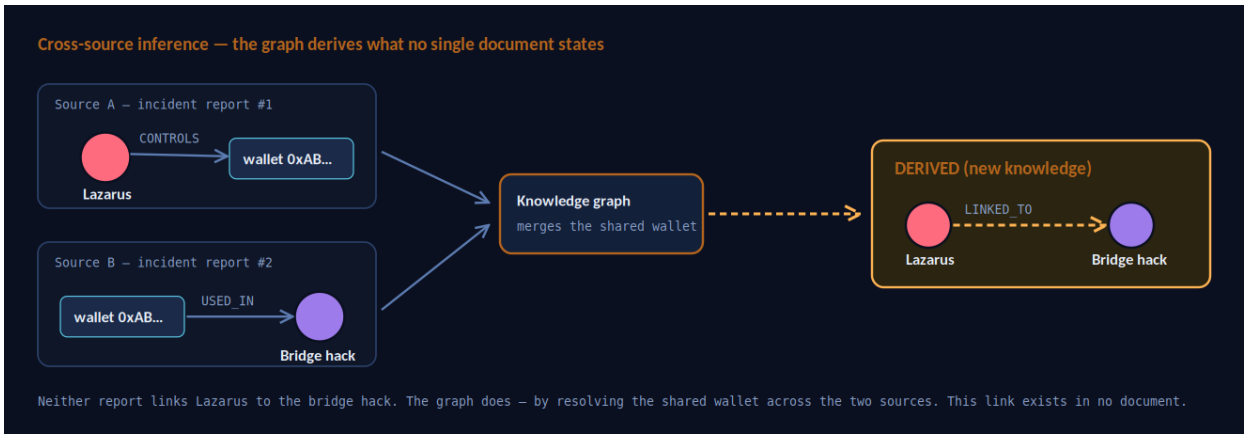

Figure 3 — Two reports never mention each other; the graph links Lazarus to a bridge hack by resolving a shared wallet across both. The derived link exists in no document.

Representative patterns: **shared infrastructure** (the same wallet across two incidents), **control gaps** (a technique used elsewhere with no mitigating control recorded for your asset), and **transitive attack paths** (a multi-hop route from a known actor to your asset through a shared dependency). This derived knowledge is the actionable intelligence the platform exists to produce, and every derived edge is explainable.

5.3 Explainable, quantified exposure for public-chain assets

Because the attack surface is transitive, a graph is its natural representation. On that structure the platform computes exposure scores deterministically, with every factor traced to edges. The number is reproducible and defensible — the property that makes it institutionally usable.

Who benefits, concretely: institutional digital-asset issuers and custodians, their CISOs and threat-intelligence teams, and the auditors and regulators who must be shown a defensible basis for risk assessments under frameworks such as MiCA and DORA.

6. The Research Problem: Construction Quality Caps Everything

The graph's reasoning is sound and deterministic — but it reasons over whatever the language model extracted. If extraction mislabels the attacker, flawless reasoning yields a confident wrong answer, and the graph gives no error. This makes construction quality the central research problem, and the system exhibits three textbook failure modes that motivate the contributions.

- **Ontology drift / class proliferation:** a free model invents a class per sentence (e.g. SecurityPatch vs SecurityPatchRelease) instead of reusing a stable vocabulary.
- **Type confusion:** events (a drain, a churn) are modelled as entity classes, collapsing the entity/event distinction temporal reasoning depends on.

- **Entity-resolution failure:** one real actor scattered across many nodes (a handle, a generic “attacker”, a wallet address), so no node accumulates the attack story.

The four contributions follow directly: (1) schema-constrained, validated construction that rejects off-schema facts; (2) entity resolution as a first-class, evaluated component; (3) graph-quality metrics as construction-failure detectors; (4) downstream-task fidelity as the real evaluation.

7. Evaluating the Ontology — A Measurement, Not an Opinion

There is no canonical “best” ontology; an ontology is a designed artefact judged by fitness for purpose. The pipeline therefore **measures** ontology fitness rather than asserting it. The `--compare-ontology` mode builds the same corpus under both an induced (LLM-designed) schema and the strict standards-grounded seed, and reports the difference.

Representative result (JPMD-on-Base corpus):

Metric	Induced (LLM)	Strict (seed)
Near-duplicate class pairs	higher	lower
Events mis-modelled as entities	present	minimal
Relation coverage	7%	47%
Largest-component ratio (cohesion)	0.33	0.93

The strict, standards-grounded ontology produces a dramatically more cohesive, better-connected graph. This is empirical evidence that schema design — not model size alone — governs graph quality, and it converts “is the ontology good?” into a reproducible measurement. The trade-off is named honestly: strict mode favours consistency across incidents over completeness on any single narrow corpus, and the coverage metric is exactly how an under-fit schema is detected.

8. Why LLM and Knowledge Graph Complement Each Other

The architecture is hybrid because the two technologies have opposite strengths. Neither alone suffices; together they cover each other's failure modes.

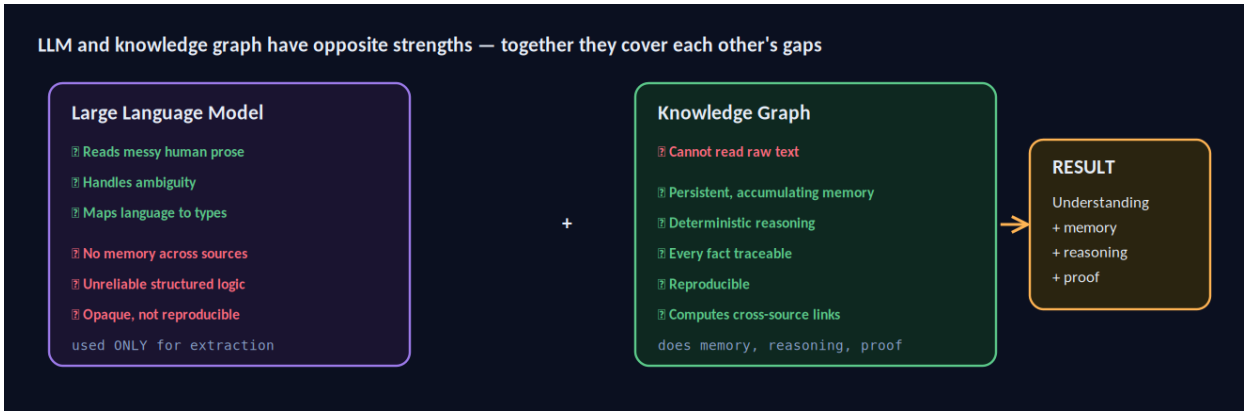

Figure 4 — The language model supplies understanding; the graph supplies memory, reasoning, and proof. The system is correct only because both are present.

The division of labour follows: the model is used only where language understanding is required (extraction); the graph is used for memory, inference, scoring, and answering. When a user asks a question, the facts are retrieved deterministically from the graph, so the model — if used to phrase the answer — cannot invent or misname entities.

8.1 The “RISC” framing as a governance property

The platform restricts the model to a small, fixed set of intelligence operations rather than open-ended generation. Framed honestly, this is a **governance and auditability property**: a bounded, reviewable operational surface, which for an institutional security function is worth more than unbounded cleverness. The term is used by analogy, not as a literal reduced-instruction-set processor.

9. Evaluation Design

The thesis substantiates each claim with measurement, not assertion:

1. **Extraction fidelity:** sample extracted triples, human-verify against their cited evidence sentences, report precision per relation type.
2. **Entity resolution:** a hand-labelled gold set of co-referent mentions; measure precision/recall against a string-matching baseline.
3. **Schema fitness:** class/relation coverage across many incidents; induced-vs-strict comparison (Section 7).
4. **Downstream-task accuracy:** a fixed battery of questions with known answers (who/what/when); measure correctness, including honest “unattributed” answers where sources do not name a culprit.
5. **Ablation:** remove each safeguard (schema constraint, verification, resolution) and quantify its individual contribution.

10. Conclusion

The knowledge graph is not one design choice among equivalent alternatives; it is the component that supplies the persistence, structure, reasoning, and explainability the institutional problem demands and that a language model cannot provide. The language model, in turn, makes building the graph from unstructured text tractable. The necessity of each is the necessity of the pair — and the engineering discipline that makes their combination reliable, measurable, and auditable is the contribution this work defends.

> *The language model supplies understanding; the knowledge graph supplies memory, reasoning, and proof. Each overcomes the other's defining limitation, and the system is correct only because both are present.*